

LKA Invoices

Architecture & source-file guide · Lingeshwar Kaparathi & Associates · GST invoice app
Live app: lingeshwarca-invoice.netlify.app · Code: github.com/adi-repo-ai/ca-invoice-app

1. The big picture

Phone / computer browser: the app (React website)

- Screens: Home, Invoices, Clients, Settings (day/night theme)
- GST maths and invoice numbers
- Makes the PDF (with UPI QR code) right in the browser
- Share / Email / WhatsApp through your own apps

↓ **sign-in & data**

FIREBASE (free Spark plan)

- **Authentication:** Google sign-in (+ email fallback)
- **Firestore:** the database
- **Security rules:** who may do what

↓ **only for "admin jobs"**

NETLIFY

- Hosts the website
- 3 small server functions: manage users, Google access check, first-admin setup
- Holds the secret Firebase admin key

GITHUB ([adi-repo-ai/ca-invoice-app](https://github.com/adi-repo-ai/ca-invoice-app)): the code lives here. A push to `main` makes Netlify rebuild the site automatically.

In one sentence: the website runs in your browser and talks directly to Firebase for sign-in and data. Firebase's **security rules** decide what each person may do. Netlify serves the website and runs three tiny server functions for jobs that need the secret admin key.

What happens when you create an invoice

1. You fill in the form. The browser works out GST and totals (`src/lib/tax.ts`).
2. **Create invoice** saves it to Firestore and takes the next number (`LKA/2026-27/000X`) in a single step called a *transaction*, so two people creating invoices at the same moment can never get the same number.
3. The security rules check everything: you're signed in, the number is the next one in order, and the invoice is now locked.
4. **Download / Email / WhatsApp** builds the PDF in your browser and hands it to your phone's share sheet, or to your email/WhatsApp app on a computer.
5. Every action is recorded in an append-only **activity log**.

How GST is calculated (`src/lib/tax.ts`)

1. Each line: **Amount = Qty × Price**. Taxable value = sum of all service lines (reimbursements excluded).
2. **Settings** → **Charge GST** switch. **OFF** (default): no GST; the PDF is a plain "INVOICE". **ON**: client in the **same state** (Telangana) → **CGST 9% + SGST 9%**; **other state** → **IGST 18%**; the PDF is a "TAX INVOICE".
3. Reimbursements (government/ROC fees) are added with **no GST**.
4. Total Invoice Value = Total Amount + GST (if on) + reimbursements, rounded to the nearest rupee ("Round off"), plus the amount in words.

2. What's stored in the database (Firestore)

Collection	Holds
<code>settings/firm</code>	Firm details, bank/UPI, GST rate, services and prices, logo, brand colour
<code>clients</code>	Saved clients (name, state, email, WhatsApp, GSTIN...)
<code>invoices</code>	Every invoice, with its own copy of client and firm details, plus status (Draft / Issued / Paid / Cancelled)
<code>counters</code>	Last invoice number used in each financial year (e.g. 2026-27 → 12)
<code>auditLog</code>	Who created, issued, paid, cancelled or sent what, and when
<code>users</code>	App users, their roles (ADMIN / STAFF) and when they were last active
<code>invites</code>	Google emails an ADMIN has given access to, until they first sign in
<code>tasks</code> , <code>events</code> , <code>notes</code> , <code>links</code>	Home-page items, private to each user

3. Every file, explained

Project configuration (root folder)

File	What it does
<code>README.md</code>	Handover guide: setup, deployment, daily use, troubleshooting
<code>package.json</code> / <code>package-lock.json</code>	List of libraries the app uses, and commands like <code>npm test</code>
<code>netlify.toml</code>	Netlify settings: how to build, where functions live, page routing, security headers
<code>firebase.json</code>	Points Firebase at the rules/indexes files; local test-database ports
<code>firestore.rules</code>	The security guard. Who can read or change what; locks issued invoices; protects numbering
<code>firestore.indexes.json</code>	Pre-built lookups Firestore needs for lists, filters and dashboard totals

<code>.env.example</code>	Template of the settings/secrets the app needs (the real <code>.env</code> is never uploaded)
<code>.gitignore</code>	Files Git must never upload (secrets, build output)
<code>index.html</code>	The single web page the app loads into
<code>vite.config.ts</code>	Settings for building the website
<code>tsconfig*.json</code>	TypeScript (type-checking) settings
<code>vitest*.config.ts</code>	Settings for the automated tests
<code>public/favicon.svg</code> , <code>robots.txt</code>	Browser-tab icon; tells search engines not to list the app

Server functions (`netlify/functions/`), the only server-side code

File	What it does
<code>admin-users.ts</code>	ADMIN-only: create users, change role, disable/enable, set password
<code>activate.ts</code>	On first Google sign-in, gives the role an ADMIN set in Settings → Users (or refuses access)
<code>claim-admin.ts</code>	One-time setup code to make the very first ADMIN from the browser
<code>_shared/admin.ts</code>	Connects to Firebase using the secret admin key stored in Netlify
<code>_shared/http.ts</code>	Checks the caller is signed in and has the right role; standard error replies

App start-up and shell (`src/`)

File	What it does
<code>main.tsx</code>	Starting point: launches the app
<code>App.tsx</code>	List of pages/addresses; sends signed-out users to login; "No access" page with setup-code box
<code>firebase.ts</code>	Connects the website to your Firebase project
<code>auth.tsx</code>	Who is signed in and their role; auto sign-out after 30 idle minutes; "last active" updates
<code>theme.ts</code>	Day / night theme (remembered per device)
<code>settings-context.tsx</code>	Loads firm settings once and shares them with every screen; applies brand colour
<code>api.ts</code>	Calls the Netlify functions with the user's sign-in token
<code>index.css</code>	Global styling (Tailwind) and the brand colour
<code>vite-env.d.ts</code>	Type definitions for the website's config values

Screens (`src/pages/`)

File	Screen
<code>Login.tsx</code>	Sign in with Google; email & password fallback with Forgot password
<code>Home.tsx</code>	Home dashboard: welcome, live date/time, quick actions, statistics, collection progress, tasks, events, notes, favourite links, overdue invoices
<code>Invoices.tsx</code>	Invoice list with filters and CSV export
<code>InvoiceEdit.tsx</code>	New invoice / edit draft: client (saved or one-off), services, reimbursements, notes, signature, totals
<code>InvoiceView.tsx</code>	One invoice: Download PDF / Email / WhatsApp, mark as paid, cancel, activity
<code>Clients.tsx</code>	Client list: search, A–Z / newest sort, total count, View and Delete
<code>ClientEdit.tsx</code>	Add/edit a client (also GSTIN / PAN / state validation)
<code>ClientView.tsx</code>	One client: details, their invoices, delete client
<code>Settings.tsx</code>	Firm details, bank/UPI, Charge GST switch, services and prices, signature, logo, colour
<code>Users.tsx</code>	Settings → Users: give Google emails access, active-now status, disable, change roles
<code>Backup.tsx</code>	Settings → Backup: download all data as JSON

Reusable pieces (`src/components/`)

File	What it is
<code>Layout.tsx</code>	Top bar with logo, menu (Home / Invoices / Clients / Settings), theme toggle, sign out; mobile menu
<code>ui.tsx</code>	Buttons, cards, form fields, alerts, switch, stat tiles, ₹ formatting, status badges
<code>Dialog.tsx</code>	In-app confirm / prompt pop-ups (replace the browser's plain ones)
<code>ThemeToggle.tsx</code>	Sun / moon button for day and night themes
<code>ClientPicker.tsx</code>	Search-and-pick saved-client dropdown
<code>ClientFields.tsx</code>	Client details form for new / one-off clients on the invoice
<code>InvoiceTable.tsx</code>	Invoice list (table on desktop, cards on phone)
<code>SettingsTabs.tsx</code>	Firm details / Users / Backup tabs

Data access (`src/data/`): reading and writing the database

File	What it does
<code>invoices.ts</code>	Create/edit drafts, issue with the next number , mark paid, cancel, lists, filters, totals
<code>clients.ts</code>	Search, add, edit and delete clients

<code>settings.ts</code>	Load and save firm settings
<code>audit.ts</code>	Write and read the activity log
<code>home.ts</code>	Your private tasks, events, notes and favourite links

Business logic (`src/lib/`): pure calculations

File	What it does
<code>tax.ts</code>	GST engine: line amounts, GST off / CGST+SGST / IGST, reimbursements, rounding
<code>fy.ts</code>	Financial year (Apr–Mar), number format LKA/2026-27/0001, Indian dates, PDF file name
<code>money.ts</code>	Paise ↔ rupees, Indian number format (1,23,456.00)
<code>words.ts</code>	Amount in words (lakh / crore)
<code>validation.ts</code>	GSTIN, PAN, IFSC, email and WhatsApp number checks
<code>states.ts</code>	Indian states with GST codes
<code>types.ts</code>	The shapes of firm, client, invoice and payment data
<code>defaults.ts</code>	Starting firm settings (visiting-card details, services, 18% GST)
<code>defaultLogo.ts</code>	The starting CA India logo image
<code>image.ts</code>	Shrinks an uploaded logo to 400px wide
<code>csv.ts</code>	Builds the CSV export file
<code>*.test.ts</code>	Automated tests for tax, words, FY numbering, money, CSV

PDF (`src/pdf/`)

File	What it does
<code>InvoicePdf.tsx</code>	The invoice PDF layout: header, bill-to, payment details + QR, table, totals, notes, signature, terms, copyright footer, blue band
<code>generate.ts</code>	Builds the UPI QR code, turns the layout into a PDF file, downloads it, names it

Scripts (`scripts/`): run by hand, not part of the website

File	What it does
<code>create-admin.ts</code>	Creates the first ADMIN from a terminal (alternative to the setup code)
<code>seed-emulator.ts</code>	Fills a local test database with demo data
<code>env.ts</code>	Helper for reading local settings

Tests (`tests/emulator/`): run against a local copy of Firebase

File	What it proves
<code>rules.test.ts</code>	Signed-out / no-role users can't access anything; issued invoices are locked; audit log can't be faked
<code>numbering.test.ts</code>	Numbers are sequential with no gaps, even with 10 invoices created at once
<code>admin-users.test.ts</code>	Only an ADMIN can manage users
<code>claim-admin.test.ts</code>	The setup code works once only
<code>activate.test.ts</code>	Only invited, verified Google emails get access
<code>helpers.ts</code> , <code>fn-helpers.ts</code>	Shared test setup

4. Where to change common things

You want to change...	Where
Firm details, bank/UPI, GST on/off, services/prices, signature, logo, colour	In the app: Settings (no code)
Who can sign in	In the app: Settings → Users (no code)
PDF look	<code>src/pdf/InvoicePdf.tsx</code>
GST maths	<code>src/lib/tax.ts</code>
Invoice number format	<code>src/lib/fy.ts</code> (and the prefix in Settings)
Who's allowed to do what	<code>firestore.rules</code> (then upload the rules to Firebase)

Deploying: a push to `main` on GitHub makes Netlify publish the website automatically. Changes to `firestore.rules` or `firestore.indexes.json` must also be uploaded to Firebase (Cloud Shell: `npx firebase deploy --only firestore:rules,firestore:indexes` , or paste the rules in Firebase console → Firestore → Rules → Publish).

LKA Invoices: architecture & source guide. Generated from the code in `adi-repo-ai/ca-invoice-app` (main).